

高程17年期末

一、简述题（本题满分 30 分）

请在试卷上答题！

1. 什么是面向对象程序设计？
2. C++ 类中 protected 访问控制的作用是什么？
3. C++ 类中虚函数的作用是什么？
4. 在 C++ 中，什么情况下两个类满足子类型关系？
5. 在 C++ 的 STL 中，迭代器的作用是什么？
6. MFC 提供的“文档 - 视”结构为程序设计带来什么好处？

二、程序分析题（本题满分 35 分）

请在试卷上答题！

(1) 指出程序的语法错误和错误原因

```
class A {
    int m1;
    const int m3;
    static int m2;
public:
    A() { m1 = 0; m3 = 100; }
    void f1() { m1 += m2+m3; }
    static void f2() { m1++; m2++; }
    void f3() const { m1 -= m2+m3; }
};
int A::m2=0;

void func(const A& x)
{
    x.f1();
    x.f2();
    x.f3();
}

int main()
{
    A a;
    a.f1();
}
```

```

    a.f3();
    a.f2();
    func(a);
    return 0;
}

```

(2) 判断程序是否存在问题，若有则指出问题

```

class A
{
public:
    int i,j;
    A() { i=j=0; }
    void f() { i++; j++; }
};

class B
{
public:
    A *p;
    B() { p = new A; }
    ~B() { delete p; p = NULL; }
    void g() { p->f(); }
};

void func(B x)
{
    x.g();
}

int main()
{
    B b1,b2;
    func(b1);
    b2 = b1;
    return 0;
}

```

(3) 写出程序运行结果

```

#include <iostream>
using namespace std;

```

```

class Passenger
{
    int num;
public:
    Passenger():num(0) { cout << "in Passenger's default
constructor\n"; }
    Passenger(int i):num(i) {cout<< "in Passenger 's par
constructor\n"; }
    Passenger (const Passenger& psg):num(psg.num) { cout << "in
Passenger's copy constructor\n"; }
    ~Passenger() { cout << "in Passenger's destructor\n"; }
};

class Car
{
public:
    int seated;
    Car(): seated(0) { cout << "in Car's default constructor\n"; }
    Car(int i): seated(i) { cout << "in Car's par constructor\n"; }
    Car(const Car& ca):seated(ca.seated) { cout << "in Car 's copy
constructor\n"; }
    ~Car() { cout << "in Car's destructor\n"; }
};

class Taxi: public Car
{
    Passenger pg;
public:
    Taxi() { cout << "in Taxi's default constructor\n"; }
    Taxi(int i): Car(i) { cout << "in Taxi's single par
constructor\n";}
    Taxi(int i, int j): Car(i), pg(j) { cout << "in Taxi's two par
constructor\n";}
    Taxi(const Taxi& tx): pg(tx.pg) { cout << "in Taxi's copy
constructor\n"; }
    ~Taxi() { cout << "in Taxi's destructor\n"; }
};

void convey(Taxi x)
{
    cout << "In convey\n";
}

int main()
{

```

```

    cout << "-----Section 1-----\n";
    Taxi t1;
    cout << "-----Section 2-----\n";
    Taxi t2(5);
    cout << "-----Section 3-----\n";
    Taxi t3(5, 7);
    cout << "-----Section 4-----\n";
    convey(t1);
    cout << "-----Section 5-----\n";
    return 0;
}

```

(4) 写出程序运行结果

```

#include <iostream>
using namespace std;

class A {
    int x,y;
public:
    A() { cout << "in A's default constructor\n"; }
    ~A() { cout<< "in A's destructor\n"; }
    void g() { cout << "in A's g\n"; }
    virtual void f() { cout << "in A's f\n"; }
};

class B: public A {
    int z;
public:
    B() { cout << "in B's default constructor\n"; }
    void f() { cout << "in B's f\n"; }
    void g() { cout << "in B's g\n"; }
    ~B() { cout << "in B's destructor\n"; }
};

void func(A *pa) {
    pa->f();
    pa->g();
}

int main()
{
    cout << "-----Section 1-----\n";
    A a;
}

```

```

func(&a);
cout << "-----Section 2-----\n";
B b;
func(&b);
cout << "-----Section 3-----\n";
A *p=new B;
func(p);
cout << "-----Section 4-----\n";
delete p;
cout << "-----Section 5-----\n";
return 0;
}

```

(5) 写出程序运行结果

```

#include <iostream>
using namespace std;

class A
{
    int m,n;
public:
    A() { m=n=0; }
    A inc1() { m++; n++; return *this; }
    A inc2() { m++; n++; return *this; }
    void dec() { m--; n--; }
    void show() { cout << m << "," << n << endl; }
};

int main()
{
    A a1;
    a1.inc1().dec();
    a1.show();

    A a2;
    a2.inc2().dec();
    a2.show();

    return 0;
}

```

三、编程实现题（本题满分 35 分）

请在试卷上答题！

1. 文件处理程序

编写程序，读取文本文件 `abc.txt`，为每一行添加行号，将结果写入 `abc2.txt`。示例：

- `abc.txt` 内容：

```
first line
second line
third line
```

- `abc2.txt` 内容：

```
1 first line
2 second line
3 third line
```

2. 实现数组类模板 `Array<T>`

不允许使用 C++ STL 容器，实现类模板 `Array<T>`，要求可兼容如下测试代码：

```
#include <iostream>
using namespace std;

int main ()
{
    int m, n;
    cin >> m;
    Array<int> a(m); // 创建包含m个int元素的数组对象a
    cin >> a;         // 输入数组每个元素的值
    cout << a.max();  // 输出数组最大元素的值
    cout << a.sum();  // 输出数组所有元素的和
    a.sort();         // 数组元素从小到大排序
    cout << a;        // 输出数组所有元素
    cin >> n;
    a.resize(n);      // 修改数组大小：n>m则尾部补0；n<m则删除尾部元素
    cout << a;        // 输出数组所有元素
    return 0 ;
}
```

3. 递归转尾递归

将如下普通递归函数，改写为功能等价的尾递归函数：

```
int recsum (int n)
{
    if (n == 1 )
        return 1 ;
    else
        return n+recsum(n-1);
}
```

参考答案

注意：由大模型生成，仅供参考！

一、简述题

1. 什么是面向对象程序设计？

面向对象程序设计（OOP）是一种以"对象"为核心的编程范式。对象是数据（属性/成员变量）与操作数据的方法（成员函数）的封装体。OOP 的四大基本特征是：

- **封装**：将数据与操作绑定在一起，隐藏内部实现细节，只通过公开接口与外界交互。
- **继承**：派生类可以复用基类的属性和方法，实现代码重用和层次化分类。
- **多态**：同一接口在不同对象中表现出不同行为（编译时多态如重载，运行时多态如虚函数）。
- **抽象**：通过类/接口定义对象的公共行为规范，忽略不相关的细节。

2. C++ 类中 `protected` 访问控制的作用是什么？

`protected` 访问控制提供介于 `private` 和 `public` 之间的访问级别：

- 类的**成员函数**和**友元**可以访问 `protected` 成员；
- 该类的**派生类**的成员函数和友元也可以访问基类的 `protected` 成员；
- 类的外部（非友元、非派生类）**不能**访问。

典型用途：基类中将希望派生类使用但不对外暴露的成员声明为 `protected`，既保证封装性，又允许派生类在继承体系中访问。

3. C++ 类中虚函数的作用是什么？

虚函数（`virtual`）用于实现**运行时多态**（动态绑定）。当通过基类指针或引用调用虚函数时，实际调用的是**指针/引用所指向对象的实际类型**中重写（`override`）的版本，而非指针/引用静态类型的版本。

实现机制：编译器为包含虚函数的类生成虚函数表（`vtable`），每个对象通过 `vptr` 指针间接调用正确的函数。这使得程序可以在运行时根据实际对象类型决定调用哪个函数，是实现"开闭原则"的关键。

4. 在 C++ 中，什么情况下两个类满足子类型关系？

两个类满足子类型关系（即"is-a"关系）的条件是：

- 派生类以 **public 继承**方式继承基类；
- 派生类满足**里氏替换原则（LSP）**：在任何需要基类对象的地方，都可以用派生类对象替代，且程序行为保持不变。

在 C++ 中，`public` 继承本身就表达了子类型关系——派生类对象可以隐式转换为基类引用/指针，基类指针可以指向派生类对象。

5. 在 C++ 的 STL 中，迭代器的作用是什么？

迭代器（`Iterator`）是 STL 的核心抽象，作用如下：

- 提供**统一的元素访问方式**：无论底层容器是 `vector`、`list`、`set` 还是 `map`，都用相同的方式（`*`、`++`、`--` 等）遍历元素；
- **解耦算法与容器**：STL 算法（如 `sort`、`find`）通过迭代器操作数据，不与具体容器绑定；
- **桥接角色**：迭代器是容器与算法之间的"粘合剂"，使算法可以通用于不同容器。

迭代器本质上是智能指针的泛化，分为输入、输出、前向、双向、随机访问五种类型。

6. MFC 提供的"文档-视"结构为程序设计带来什么好处？

MFC 的 Document-View 架构将数据与显示分离，主要好处：

- **数据与表现分离**：文档（`Document`）负责数据存储和处理，视图（`View`）负责数据显示和用户交互，两者独立变化；

- **多视图支持**：一个文档可以对应多个视图（如电子表格同时显示表格和图表），各视图自动保持同步；
 - **便于维护和扩展**：修改显示逻辑不影响数据处理逻辑，符合单一职责原则；
 - **框架自动化**：MFC 自动管理文档的序列化（保存/加载）、视图更新通知等，减少样板代码。
-

二、程序分析题

(1) 指出程序的语法错误和错误原因

错误①： `const int m3` 在构造函数体内赋值

```
A() { m1 = 0; m3 = 100; } // 错误
```

`const` 成员变量必须在**构造函数初始化列表**中初始化，不能在函数体内赋值。正确写法：

```
A() : m1(0), m3(100) {}
```

错误②： `static void f2()` 访问非静态成员 `m1`

```
static void f2() { m1++; m2++; } // m1++ 错误
```

静态成员函数没有 `this` 指针，无法访问非静态成员变量 `m1`。只能访问静态成员 `m2`。

错误③： `void f3() const` 修改成员变量 `m1`

```
void f3() const { m1 -= m2+m3; } // 错误
```

`const` 成员函数承诺不修改对象状态，但函数体内修改了 `m1`，违反 `const` 语义。

错误④： `func(const A& x)` 中调用非 `const` 成员函数

```
void func(const A& x) {  
    x.f1();    // 错误：f1() 不是 const 成员函数  
    x.f2();    // 正确：静态成员函数不涉及 this  
    x.f3();    // 正确：f3() 是 const 成员函数  
}
```

const 引用只能调用 const 成员函数，f1() 不是 const，因此 x.f1() 编译错误。

(2) 判断程序是否存在问题，若有则指出问题

存在问题，涉及浅拷贝导致的双重释放和悬空指针。

问题① — 浅拷贝导致悬空指针：

func(b1) 按值传递 B 对象。由于类 B 未定义拷贝构造函数，编译器生成的默认拷贝构造函数进行**浅拷贝**，使得形参 x.p 和 b1.p 指向同一块动态内存。当 func 返回时，形参 x 被销毁，其析构函数 delete p 释放了那块内存，导致 b1.p 成为**悬空指针**（dangling pointer）。

问题② — 双重释放（double free）：

b2 = b1 使用默认赋值运算符（也是浅拷贝），使得 b2.p 也指向同一块内存（且 b2 原来持有的内存泄漏）。程序结束时 b1 和 b2 各自调用析构函数，对同一地址执行两次 delete，导致**未定义行为**（通常程序崩溃）。

解决方案：为类 B 实现深拷贝的拷贝构造函数和赋值运算符（或禁止拷贝），遵循"三法则"（Rule of Three）。

(3) 写出程序运行结果

```
-----Section 1-----
in Car's default constructor
in Passenger's default constructor
in Taxi's default constructor
-----Section 2-----
in Car's par constructor
in Passenger's default constructor
in Taxi's single par constructor
-----Section 3-----
in Car's par constructor
in Passenger 's par constructor
in Taxi's two par constructor
-----Section 4-----
in Car's default constructor
in Passenger's copy constructor
in Taxi's copy constructor
In convey
in Taxi's destructor
in Passenger's destructor
```

```
in Car's destructor
-----Section 5-----
in Taxi's destructor
in Passenger's destructor
in Car's destructor
in Taxi's destructor
in Passenger's destructor
in Car's destructor
in Taxi's destructor
in Passenger's destructor
in Car's destructor
```

分析要点：

- 构造顺序：基类 → 成员对象（声明顺序） → 派生类自身
 - 析构顺序：与构造严格相反
 - **Section 4：** Taxi 的拷贝构造函数初始化列表只写了 `pg(tx.pg)`，未显式调用 Car 的拷贝构造函数，因此基类 Car 部分使用**默认构造函数**（而非拷贝构造），然后 Passenger 用拷贝构造，最后执行 Taxi 拷贝构造函数体。
 - **Section 5：** main 结束，按 `t3 → t2 → t1` 逆序析构。
-

(4) 写出程序运行结果

```
-----Section 1-----
in A's default constructor
in A's f
in A's g
-----Section 2-----
in A's default constructor
in B's default constructor
in B's f
in A's g
-----Section 3-----
in A's default constructor
in B's default constructor
in B's f
in A's g
-----Section 4-----
in A's destructor
-----Section 5-----
in B's destructor
```

```
in A's destructor  
in A's destructor
```

分析要点：

- `f()` 是虚函数 → **动态绑定**，`pa->f()` 调用指针实际指向对象的类型中的版本。
 - Section 1: `pa` 指向 `A` → `A::f()`
 - Section 2: `pa` 指向 `B` → `B::f()`
 - Section 3: `pa` 指向 `B` (`new B`) → `B::f()`
- `g()` 是非虚函数 → **静态绑定**，`pa->g()` 始终调用 `A::g()` (取决于指针类型 `A*`)。
- **Section 4 陷阱**：`delete p` (`A*` 指向 `B`) ——由于 `~A()` **不是虚析构函数**，`delete` 只调用 `~A()` 而**不会调用** `~B()`，这是未定义行为（实际常见表现为只输出 `in A's destructor`，`B` 的资源泄漏）。这也说明了**基类析构函数应声明为 `virtual`** 的重要性。
- Section 5：局部对象 `b` 和 `a` 正常析构。

(5) 写出程序运行结果

```
1,1  
1,1
```

分析要点：

- `inc1()` 和 `inc2()` 的返回类型是 `A` (**值返回**，不是引用 `A&`)。
- `a1.inc1()` 先将 `a1` 的 `m` 和 `n` 从 `(0,0)` 自增为 `(1,1)`，然后**返回** `a1` 的**临时拷贝**。
- `.dec()` 作用在**临时拷贝**上，将其 `m` 和 `n` 减为 `(0,0)`，然后临时对象被销毁。
- `a1.show()` 输出的是原对象 `a1`，它仍是 `(1,1)`。
- `a2` 同理，`inc1` 和 `inc2` 逻辑完全一致。
- 若要让链式调用真正作用在原对象上，应改为返回引用：`A& inc1() { m++; n++; return *this; }`，此时输出将是 `0,0` 和 `0,0`。

三、编程实现题

1. 文件处理程序

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream infile("abc.txt");
    if (!infile) {
        cerr << "Cannot open abc.txt" << endl;
        return 1;
    }

    ofstream outfile("abc2.txt");
    if (!outfile) {
        cerr << "Cannot create abc2.txt" << endl;
        return 1;
    }

    string line;
    int lineNum = 1;
    while (getline(infile, line)) {
        outfile << lineNum << " " << line << endl;
        lineNum++;
    }

    infile.close();
    outfile.close();
    return 0;
}
```

2. 实现数组类模板 `Array<T>`

```
#include <iostream>
using namespace std;

template <typename T>
class Array {
    T* data;
    int len;

public:
    // 构造函数：创建包含 m 个元素的数组（值初始化）
```

```

Array(int m) : len(m) {
    data = new T[m]();
}

// 析构函数
~Array() { delete[] data; }

// 拷贝构造函数（深拷贝）
Array(const Array& other) : len(other.len) {
    data = new T[len];
    for (int i = 0; i < len; i++)
        data[i] = other.data[i];
}

// 赋值运算符（深拷贝）
Array& operator=(const Array& other) {
    if (this != &other) {
        delete[] data;
        len = other.len;
        data = new T[len];
        for (int i = 0; i < len; i++)
            data[i] = other.data[i];
    }
    return *this;
}

// 输入运算符 >>
friend istream& operator>>(istream& in, Array& a) {
    for (int i = 0; i < a.len; i++)
        in >> a.data[i];
    return in;
}

// 输出运算符 <<
friend ostream& operator<<(ostream& out, const Array& a) {
    for (int i = 0; i < a.len; i++)
        out << a.data[i] << " ";
    return out;
}

// 返回最大值
T max() const {
    T m = data[0];
    for (int i = 1; i < len; i++)
        if (data[i] > m)

```

```

        m = data[i];
    return m;
}

// 返回所有元素之和
T sum() const {
    T s = T(); // 零初始化
    for (int i = 0; i < len; i++)
        s += data[i];
    return s;
}

// 冒泡排序（升序）
void sort() {
    for (int i = 0; i < len - 1; i++)
        for (int j = 0; j < len - 1 - i; j++)
            if (data[j] > data[j + 1]) {
                T temp = data[j];
                data[j] = data[j + 1];
                data[j + 1] = temp;
            }
}

// 调整数组大小：n>len 则尾部补 T(); n<len 则截断尾部
void resize(int n) {
    T* newData = new T[n](); // 新数组，值初始化（尾部自动补0）
    int copyCount = (n < len) ? n : len;
    for (int i = 0; i < copyCount; i++)
        newData[i] = data[i];
    delete[] data;
    data = newData;
    len = n;
}
};

```

3. 递归转尾递归

尾递归的核心思想：将中间结果通过**累积参数**传递到下一次递归调用，使得递归调用是函数的最后一个操作（无需在返回后做额外计算）。

```

// 对外接口保持原函数名，内部调用辅助尾递归函数
int recsum(int n) {

```

```
        return recsum_tail(n, 0);
    }

    // 尾递归辅助函数: acc 累积中间和
    int recsum_tail(int n, int acc) {
        if (n == 0)
            return acc;
        else
            return recsum_tail(n - 1, acc + n);
    }
}
```

等价性说明:

- 原函数 $\text{recsum}(5) = 5 + 4 + 3 + 2 + 1 = 15$
- 尾递归版 $\text{recsum_tail}(5, 0) \rightarrow \text{recsum_tail}(4, 5) \rightarrow \text{recsum_tail}(3, 9) \rightarrow \text{recsum_tail}(2, 12) \rightarrow \text{recsum_tail}(1, 14) \rightarrow \text{recsum_tail}(0, 15) \rightarrow \text{返回 } 15$
- 最后一个操作是纯递归调用，编译器可优化为迭代，避免栈溢出。